

Strings in Java

Objectives

- String, StringBuilder
- Many methods, explore!
- Conversions
- Convenience syntax
- Regular expressions
- Formatting

Strings and the Java Virtual Machine

- Working with Strings can seem intuitive

```
String s= "Fred";  
s= s+ "Bloggs";  
System.out.println ( "Hello "+ s );
```

- However, beneath the surface it is more sophisticated

Strings are Objects

- Strings in Java are objects and are immutable
- String objects are available in Java through a **String pool** in the runtime environment
- When working with Strings, you can choose whether to use the pool

```
String name1 = "hello";           // use one from the pool
String name2 = "hello";           // use the same one from the pool
String name3 = new String("hello"); // make a brand new one!
boolean same = (name1 == name2);   // this will be true
boolean sameAgain = (name1 == name3); // this will be false!
```

Comparing Strings

- Strings should *never* be compared using `==`
- Strings should *always* be compared using the `equals` method
- To avoid unexpected null pointers, call equals on string literals where possible

```
String name1 = "hello";           // use one from the pool
String name3 = new String("hello"); // make a brand new one!

boolean betterApproach = (name1.equals(name3)); // true

boolean mightGetNullPointer= (name1.equals("hello")); // name1 could be null
boolean noNullPointer = ("hello".equals(name3)); // String literal cannot be null
```

Common String Methods

- The String class provides a number of useful methods for use when working with Strings
- Commonly used String methods include
 - `int length()`
 - `char charAt(n)`
 - `int indexOf ( char/String [,from] )`
 - `String substring ( begin [,end] )`

```
String s2, s= "substantial";  
s2= s.substring ( 3, 7 ); // "stan"
```

Additional String Methods

- boolean equals (s2)
- int compareTo (s2)
- String toUpperCase ()
- String trim()
- boolean beginsWith (String)
- boolean contains (String)
- String concat (String)
- String replace (old, new)
 - Overloads accept character or String arguments
 - Method returns a new String object

String Immutability

- Once a String has been created, it cannot be changed

```
String s= "Fred"; // This string literal is fixed now  
s= s+ "Bloggs"; // A new String had to be created here
```

- Any String manipulation methods return a new String object
 - This has performance implications for strings that change often
- Therefore, for Strings that change often, different classes should be used

Alternative Classes

- There are three different classes available when working with Strings
 - **String** - Flexible but cannot be changed
 - **StringBuilder** - Faster, changeable
 - **StringBuffer** - Thread safe StringBuilder

The StringBuilder Class

- Construction
 - From existing String, or specify capacity
- Methods include
 - `StringBuilder append ( <various> )`
 - `void setCharAt ( pos, char )`
 - `StringBuilder insert ( pos, <various> )`
 - `StringBuilder deleteCharAt ( pos )`
 - `StringBuilder delete ( start, end )`
 - `StringBuilder replace ( start, end, String )` - replace a range of characters with the supplied String
 - `int capacity() - cf length()`

Conversion to Strings

- Between the string classes
 - Constructors
 - `StringBuilder.toString`
 - `String.getChars()`
- From primitives and Objects
 - `static String String.valueOf( <various> )`
 - `static String String.valueOf( Object )`
 - Calls the Object's `toString()` method
 - Null-safe

Conversion from Strings

- To convert Strings to integers and doubles and the other primitives, the wrapper classes (discussed in the Collections chapter) provide convenient static **parseX** functions
 - Integer.parseInt()
 - Double.parseDouble()
 - Float.parseFloat()

```
String integerAsString = "1";  
int intValue = Integer.parseInt(integerAsString);  
String doubleAsString = "1.6";  
double doubleValue = Double.parseDouble(doubleAsString);
```

Automatic Translations

- The compiler translates “convenience syntax”
 - `s = “Fred”` creates a new String. No need for “new”
 - `'+'` and `'+='` operators call `StringBuilder.append()`
- Hence these shortcuts all work:

```
String s= "Fred";  
s+= " Bloggs";  
//Equivalent to: new StringBuilder("Result is ").append( String.valueOf(x) ) )  
System.out.println ( "Result is "+ x );
```

Regular Expressions

- Java 5 introduces more methods such as:
 - boolean `String.matches ( String regex )`
 - `String[ ] split ( String regex )`
 - `String.format ( format-string, args )`

```
String s= "47-40-97";  
if (s.matches("[0-9]{2}+-[0-9]{2}+-[0-9]{2}+")) // Is it a sort code?  
    String[] ss= s.split("-"); // Get each segments of the sort code.  
s= String.format( "%s-%s-%s", ss[0], ss[1], ss[2]); // Re-assemble sort code
```

Formatting

- The **java.text** package provides many **Format** classes
 - Dates, Currencies, Numbers
- Below are examples using the **SimpleDateFormat**

```
Format yearOnly = new SimpleDateFormat("yyyy");  
Format dayMonthYear = new SimpleDateFormat("dd MM yyyy");  
Format time = new SimpleDateFormat("hh mm");  
System.out.println(yearOnly.format(new Date()));  
System.out.println(dayMonthYear.format(new Date()));  
System.out.println(time.format(new Date()));
```

```
2018  
01 05 2018  
09 34
```

Summary

- String, StringBuilder
- Many methods, explore!
- Conversions
- Convenience syntax
- Regular expressions
- Formatting